



Universidad Tecnológica de Panamá

Centro Regional de Chiriquí

Facultad de Ingeniería de Sistemas Computacional

Licenciatura en Gestión y Desarrollo de Software

Documento:

Apache Hive

Integrantes:

Edgar Atencio
Yohana Franceschi
Isaac Carrillo
Gregory Fernández

Semestre I

2026

Tabla de contenido

1. ¿Qué es Apache Hive?
2. ¿Qué problema resuelve?
3. Arquitectura y componentes
4. HiveQL y técnicas de optimización
5. Criterios de uso
6. Aclaración: el «otro» Hive (Flutter/Dart)
7. Referencias

1. ¿Qué es Apache Hive?

Apache Hive es un sistema de **data warehouse distribuido** construido sobre Apache Hadoop. Permite consultar y analizar grandes volúmenes de datos —de terabytes a petabytes— mediante **HiveQL**, un lenguaje declarativo muy parecido a SQL.

Origen: lo creó el equipo de ingeniería de Facebook/Meta hacia 2010, con el objetivo de que analistas con base en SQL pudieran trabajar con Big Data sin escribir lógica de bajo nivel. La rama estable vigente es Hive 4.

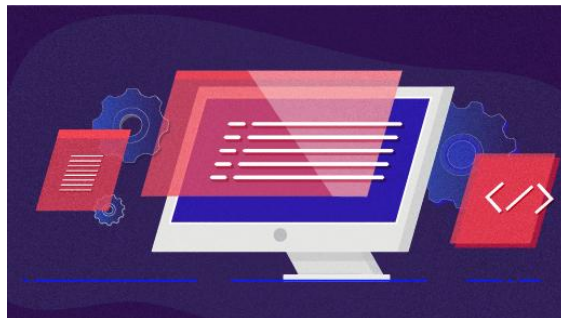


2. ¿Qué problema resuelve?

Antes de Hive, analizar datos a gran escala en Hadoop obligaba a escribir trabajos MapReduce en Java. Eso implicaba dos problemas:

- Tiempos de desarrollo altos y una curva de aprendizaje empinada.
- Código rígido y difícil de mantener.

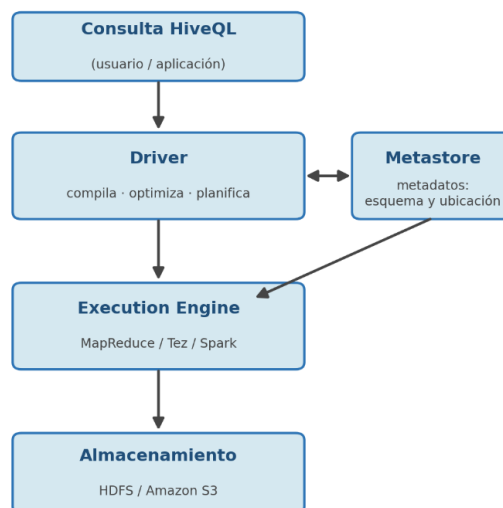
Hive añade una capa de abstracción relacional sobre el almacenamiento distribuido y abre el Big Data a un público más amplio. Está optimizado para **procesamiento por lotes (batch)** y analítica (**OLAP**), no para cargas transaccionales de baja latencia (**OLTP**).



3. Arquitectura y componentes

La arquitectura de Hive se apoya en cuatro componentes que trabajan de forma coordinada:

Componente	Función principal
Metastore	Repositorio central de metadatos: esquema de las tablas, columnas, tipos de datos y ubicación física de los archivos. Es el corazón operativo de Hive.
Driver	Gestiona el ciclo de vida de la consulta: recibe la sentencia HiveQL, la compila, la optimiza y genera el plan de ejecución.
Execution Engine	Ejecuta el plan generado por el Driver. En las primeras versiones usaba MapReduce; hoy soporta motores más rápidos como Apache Tez o Apache Spark.
Almacenamiento (HDFS / S3)	Capa física donde residen los datos. Normalmente HDFS, aunque es compatible con almacenamiento en la nube como Amazon S3.



Flujo: la consulta HiveQL se compila en el Driver (validando esquema y ubicación contra el Metastore), el Execution Engine la ejecuta de forma distribuida y los datos residen en el almacenamiento subyacente.

4. HiveQL y técnicas de optimización

HiveQL replica gran parte de la sintaxis de SQL, pero la naturaleza batch del sistema impone diferencias. Históricamente Hive no permitía modificaciones a nivel de fila; las versiones modernas (Hive 3.x en adelante) incorporaron soporte **ACID** mediante tablas transaccionales en formato ORC, habilitando INSERT, UPDATE, DELETE y MERGE. Aun así, ese soporte está pensado para correcciones y cargas controladas por lotes, no para alta concurrencia transaccional.

Técnicas clave para rendimiento a gran escala:

- **Partitioning (particionado):** segmenta la tabla por los valores de una o más columnas (p. ej. año, mes, región). El motor omite directorios completos y escanea solo lo necesario.
- **Bucketing (distribución por bloques):** subdivide los datos en archivos de tamaño fijo aplicando una función hash sobre una columna. Optimiza joins y muestreos.
- **Formatos columnares:** pasar de texto plano (CSV) a ORC o Parquet reduce el almacenamiento y acelera la lectura.

Ejemplo en HiveQL:

```
-- Tabla particionada por anio
CREATE TABLE ventas (
  producto STRING,
  monto     DOUBLE
)
PARTITIONED BY (anio INT);

-- Cargar datos en una particion concreta
INSERT INTO ventas PARTITION (anio = 2025)
VALUES ('Laptop', 1200.0), ('Mouse', 25.0);

-- Consulta analitica: total vendido por anio
SELECT anio, SUM(monto) AS total
FROM ventas
GROUP BY anio;
```

5. Criterios de uso

Recomendado (Sí):

- ETL por lotes a gran escala.
- Construcción y explotación de Data Lakes empresariales.
- Reportes históricos de Business Intelligence donde la latencia no es crítica.

No recomendado (NO):

- Aplicaciones interactivas o servicios web que exijan respuestas en milisegundos.
- Sistemas con transaccionalidad concurrente en tiempo real.
- Volúmenes pequeños, donde una base relacional (PostgreSQL, MySQL) es más simple y rápida.

***Ecosistema:** Spark SQL, Presto/Trino e Impala resuelven problemas analíticos similares con foco en interactividad en memoria, y a menudo reutilizan el Metastore de Hive.*

6. Aclaración: el «otro» Hive (Flutter/Dart)

Conviene distinguir Apache Hive de un paquete homónimo del ecosistema **Flutter/Dart**, llamado simplemente Hive: una base de datos local NoSQL (clave-valor), ligera, para persistir datos en el dispositivo cliente. **No comparte código, arquitectura ni infraestructura con Apache Hive; solo el nombre.** En prototipos se usa a pequeña escala para ilustrar persistencia clave-valor, por lo que conviene evitar confundir ambas herramientas al documentar soluciones.

7. Referencias

- Apache Software Foundation. *Apache Hive — Documentación oficial*. <https://hive.apache.org/docs/latest/>
- Apache Software Foundation. *Introduction to Apache Hive*. <https://hive.apache.org/docs/latest/introduction-to-apache-hive/>
- Amazon Web Services. *¿Qué es Apache Hive?*. <https://aws.amazon.com/what-is/apache-hive/>
- Apache Software Foundation. *Apache Hive — Quickstart*. <https://hive.apache.org/development/quickstart/>